

CSC13009 – Mobile Application Development

Seminar

I. Information

- Group ID: 01
- Group name: PandQ
- Members: 5

ID	Student ID	Full name
1	22120280	Phan Hồng Phúc
2	22120285	Nguyễn Văn Phước
3	22120288	Đỗ Thị Kim Phượng
4	22120290	Lê Minh Quân
5	22120296	Lê Văn Quang

II. Nội dung

1. Giới thiệu

Jetpack Compose là một bộ công cụ hiện đại giúp xây dựng giao diện người dùng gốc của Android. Jetpack Compose đơn giản hoá và đẩy nhanh quá trình phát triển giao diện người dùng trên Android nhờ mã ngắn gọn, các công cụ mạnh mẽ và API Kotlin trực quan.

Trước đây, UI được khai báo trong XML, nhưng khi dữ liệu thay đổi, lập trình viên phải tự tay cập nhật UI bằng các lệnh. Việc phải đồng bộ thủ công trạng thái và giao diện này là nguyên nhân lớn gây ra lỗi

Compose sử dụng mô hình khai báo. Bạn mô tả giao diện trông như thế nào với một trạng thái dữ liệu nhất định, thay vì mô tả cách tạo ra nó

- Khi trạng thái thay đổi, một giao diện mới cho trạng thái đó được tạo ra
- Compose đủ thông minh để bỏ qua những thành phần không thay đổi, giúp tối ưu hiệu suất

2. Kiến trúc & nguyên lý cốt lõi

Jetpack Compose không chỉ là một thư viện UI mà là một sự thay đổi toàn diện về mô hình kiến trúc trong phát triển Android. Dựa trên tài liệu "Jetpack Compose Pathway"[1], "Testing in Compose" và "Building Adaptive Apps"[5] từ Google, kiến trúc của dự án được xây dựng dựa trên 5 trụ cột chính sau:

2.1. Tư duy Declarative UI và Composable Functions[1]

Khác với phương pháp Imperative (Mệnh lệnh) truyền thống nơi lập trình viên thao tác trực tiếp lên cây View (DOM/View hierarchy), Compose sử dụng mô hình Declarative (Khai báo).

- Annotation `@Composable`: Đây là thành phần cơ bản nhất. Bất kỳ hàm nào được đánh dấu bằng `@Composable` sẽ được trình biên dịch Compose (Compose Compiler) xử lý đặc biệt để chuyển đổi dữ liệu thành cây UI.

- Tính chất của Composable:

- **Idempotent (Lũy đẳng)**: Với cùng một dữ liệu đầu vào, hàm Composable luôn trả về cùng một kết quả UI, giúp dễ dàng kiểm thử và dự đoán hành vi.
- **Side-effect free (Không có tác dụng phụ)**: Các hàm này lý tưởng nhất là không nên thực hiện các tác vụ nặng (như gọi API, đọc database) trực tiếp trong thân hàm, mà phải thông qua các cơ chế quản lý Side-effect (như `LaunchedEffect`).
- **UI as Code**: Giao diện được xây dựng hoàn toàn bằng Kotlin, cho phép sử dụng đầy đủ sức mạnh của ngôn ngữ (if/else, loops) để điều khiển logic hiển thị linh hoạt hơn XML.

2.2. Cơ chế Recomposition (Tái tổ hợp) thông minh [1]

Recomposition là quá trình gọi lại các hàm Composable khi trạng thái (State) của ứng dụng thay đổi. Đây là cơ chế cốt lõi giúp UI luôn đồng bộ với dữ liệu.

- **Cập nhật thông minh**: Compose theo dõi các biến State. Khi một biến thay đổi, Compose chỉ vẽ lại (recompose) những thành phần UI thực sự sử dụng biến đó và bỏ qua các thành phần không bị ảnh hưởng.
- **Tối ưu hiệu năng**: Hệ thống có khả năng chạy song song các hàm Composable hoặc hủy bỏ việc vẽ lại nếu dữ liệu thay đổi quá nhanh (ví dụ: đang gõ phím liên tục), đảm bảo ứng dụng luôn mượt mà (60fps/120fps).

2.3. State Management & Unidirectional Data Flow (UDF) [1]

Kiến trúc quản lý trạng thái của dự án tuân theo nguyên tắc Unidirectional Data Flow (Luồng dữ liệu một chiều), giúp tách biệt rõ ràng giữa logic và giao diện.

- **State:** Dữ liệu trạng thái (ví dụ: danh sách sản phẩm, trạng thái loading) đi từ trên xuống dưới (State flows down) từ ViewModel xuống các Composable.
- **Events:** Sự kiện tương tác (ví dụ: click nút "Mua ngay") đi từ dưới lên trên (Events flow up), gọi vào các hàm xử lý trong ViewModel.
- **State Hoisting (Đẩy trạng thái lên cao):** Đây là mẫu thiết kế (design pattern) quan trọng trong Compose. Trạng thái được chuyển ra khỏi Composable con và được truyền vào qua tham số. Điều này giúp Composable trở nên "Stateless" (không trạng thái), dễ dàng tái sử dụng và kiểm thử (Unit Test).

2.4. Semantics Tree & Testing Architecture [1]

Theo tài liệu về "Testing in Compose", kiến trúc của Compose tạo ra hai cây song song:

- Composition UI Tree: Cây dùng để hiển thị hình ảnh lên màn hình.
 - Semantics Tree: Cây ngữ nghĩa mô tả ý nghĩa của UI (ví dụ: nút này có nhãn là "Login", trạng thái là "Disabled").
- Vai trò: Hệ thống Testing và các dịch vụ hỗ trợ người khuyết tật (Accessibility) tương tác với Semantics Tree chứ không phải UI Tree.
 - Ứng dụng trong dự án: Việc hiểu rõ Semantics Tree giúp nhóm phát triển viết các bài kiểm thử giao diện (UI Tests) chính xác bằng cách sử dụng các hàm như `composeTestRule.onNodeWithText(...)` để tìm và tương tác với các phần tử, đảm bảo chất lượng ứng dụng trước khi release.

2.5. Adaptive Layouts (Giao diện thích ứng) [5]

Dựa trên hướng dẫn "Build adaptive apps", dự án không chỉ thiết kế cho điện thoại mà còn sẵn sàng cho các thiết bị màn hình lớn (Tablet, Foldable).

- Window Size Classes: Sử dụng thư viện material3-window-size-class để phân loại màn hình thành 3 nhóm: Compact (điện thoại dọc), Medium (máy tính bảng nhỏ/điện thoại gấp), và Expanded (máy tính bảng lớn).
- Responsive Composable: Thay vì tạo nhiều file layout riêng biệt (như layout-sw600dp trong XML), Compose cho phép thay đổi bố cục ngay trong code dựa trên Size Class hiện tại.

Ví dụ: Sử dụng **“NavigationRail”** cho màn hình rộng thay vì **“BottomNavigation”** để tận dụng không gian ngang tốt hơn.

3. So sánh Jetpack Compose vs XML

3.1 Bảng so sánh

Tiêu chí	XML	Jetpack Compose
Mô hình lập trình	Imperative (Mệnh lệnh): Bạn phải tự tay thay đổi UI (ví dụ: <code>textView.setText("Hello")</code>). Bạn quản lý cách UI thay đổi.	Declarative (Khai báo): Bạn mô tả UI trông như thế nào với một trạng thái nhất định. Khi trạng thái đổi, UI tự vẽ lại. Công thức: $UI = f(State)$
Ngôn ngữ sử dụng	Hỗn hợp: Sử dụng XML để dựng Layout và Kotlin/Java để xử lý Logic. Phải chuyển đổi ngữ cảnh liên tục.	100% Kotlin: UI và Logic đều viết bằng Kotlin. Tận dụng được sức mạnh của ngôn ngữ (If/Else, For loop ngay trong UI).
Độ dài mã nguồn	Rườm rà (High Verbosity): Cần nhiều file (Layout, Adapter, ViewHolder, Styles). Nhiều "Boilerplate code" (code mẫu lặp lại).	Ngắn gọn (Concise): Giảm thiểu code đáng kể. Không cần Adapter hay ViewHolder cho List. UI được chia nhỏ thành các hàm (Composable).
Quản lý trạng thái	Dễ lỗi: Trạng thái nằm ở cả View và Code (ví	Single Source of Truth: Trạng thái được đẩy

	dự: Checkbox tự lưu trạng thái checked). Dễ gây ra lỗi bất đồng bộ dữ liệu.	lên trên (State Hoisting). View chỉ hiển thị dữ liệu được truyền vào, không tự lưu trạng thái.
Hiệu năng	Layout Inflation: Tốn kém tài nguyên khi phải đọc và phân tích file XML lúc runtime. Nesting Layout (layout lồng nhau) làm giảm FPS.	Code Compiled: UI là các hàm Kotlin được biên dịch sẵn. Layout phẳng (flat hierarchy), cơ chế Recomposition thông minh chỉ vẽ lại phần thay đổi.
Khả năng tùy biến	Khó: Phải kế thừa View , ViewGroup , đặng tới Canvas , onMeasure , onLayout rất phức tạp.	Linh hoạt: Mọi thứ đều là Composable function. Dễ dàng lồng ghép, bọc (wrap) các view để tạo UI phức tạp.
Công cụ hỗ trợ	Layout Editor (Design): Kéo thả trực quan rất mạnh, xem trước ngay lập tức mà không cần build nhiều.	@Preview: Cho phép xem trước UI với nhiều dữ liệu mẫu, chế độ sáng/tối. Tuy nhiên cần thời gian build lại khi sửa code (đang cải thiện)
Độ khó khi học	Dễ tiếp cận ban đầu: Tư duy kiểu "tìm view và sửa" khá trực quan cho người mới. Nhưng khó để thành thạo (Custom View).	Shift tư duy: Cần thay đổi tư duy sang "Khai báo". Ban đầu hơi trừu tượng, nhưng khi hiểu State thì phát triển cực nhanh.

3.2 Các trường hợp sử dụng: Khi nào nên sử dụng XML hoặc Jetpack Compose?

Khi nào nên sử dụng XML:

- Dự án kế thừa: Nếu bạn đang duy trì hoặc cải tiến một dự án hiện có sử dụng nhiều XML, thì việc tiếp tục sử dụng XML có thể hợp lý, đặc biệt nếu bạn đang hỗ trợ nhiều phiên bản hệ điều hành Android.
- Nhóm cộng tác: Khi làm việc trong các nhóm lớn, nơi các nhà thiết kế, nhà phát triển và các bên liên quan khác cần cộng tác, XML cung cấp khả năng phân tách mối quan tâm rõ ràng, có thể đơn giản hóa giao tiếp và quy trình làm việc.
- Yêu cầu về công cụ: Nếu bạn dựa vào các công cụ như Trình chỉnh sửa bố cục, có giao diện kéo và thả, thì XML có thể phù hợp hơn.

Khi nào nên sử dụng Jetpack Compose:

- Dự án mới: Đối với các dự án mới, đặc biệt là các dự án hướng đến các phiên bản Android hiện đại, Compose là một lựa chọn tuyệt vời. Kiến trúc hiện đại và phương pháp Kotlin-first của nó giúp Compose lý tưởng để xây dựng giao diện người dùng động hiệu quả hơn.
- Giao diện người dùng động: Nếu ứng dụng của bạn yêu cầu giao diện người dùng phức tạp và động, phương pháp phản ứng của Jetpack Compose sẽ đơn giản hóa việc quản lý và cập nhật trạng thái giao diện người dùng.
- Năng suất của nhà phát triển: Compose có thể giảm đáng kể thời gian phát triển bằng cách loại bỏ mã mẫu, lý tưởng cho các nhóm muốn tăng hiệu quả.

4. Các thành phần quan trọng trong Compose

Jetpack Compose cung cấp một tập hợp các Material Components, những thành phần giao diện người dùng tuân theo chuẩn Material Design, giúp xây dựng giao diện nhất quán, hiện đại và dễ tùy biến.

Compose cho phép sử dụng các composable sẵn có để triển khai các thành phần UI, nhờ đó không cần định nghĩa thủ công mọi styling hay layout, giúp tiết kiệm thời gian và đảm bảo giao diện tuân theo chuẩn thiết kế.

Trong phần này, ta sẽ đi sâu vào 4 nhóm thành phần cốt lõi:

4.1 Layouts

Layouts trong Jetpack Compose cung cấp những khối xây dựng cơ bản để tổ chức và sắp xếp thành phần giao diện trên màn hình, cơ bản như:

- Column, Row, Box:
 - Đây là ba thành phần bố cục cơ bản nhất, thay thế cho các Layout phức tạp như LinearLayout hay FrameLayout trong hệ thống View truyền thống.
 - Column sắp xếp các thành phần con theo chiều dọc.
 - Row sắp xếp các thành phần con theo chiều ngang.
 - Box dùng để xếp chồng các thành phần lên nhau (ví dụ: hiển thị icon trên một bức ảnh nền).
- Modifier:
 - Modifier là một đối tượng chuỗi (chainable) mạnh mẽ cho phép tùy chỉnh giao diện và hành vi của bất kỳ Composable nào. Nó là yếu tố thống nhất trong Compose, thay thế cho hàng chục thuộc tính XML rải rác.
 - Các chức năng phổ biến:
 - padding(): Thêm khoảng đệm.
 - background(): Đặt màu nền.
 - clickable(): Xử lý sự kiện nhấp chuột.
 - size(): Xác định kích thước cố định.
 - fillMaxWidth(), wrapContentHeight(): Xác định kích thước linh hoạt.
 - Dưới đây là một ví dụ dùng Modifier để format một Composable:

```
@Composable
private fun Greeting(name: String) {
    Column(
        modifier = Modifier
            .padding(24.dp)
            .fillMaxWidth()
    ) {
        Text(text = "Hello, ")
        Text(text = name)
    }
}
```

- Trong đoạn code trên ta có sử dụng padd

4.2 Basic UI Components

Compose cung cấp sẵn các Composable cơ bản nhất để hiển thị nội dung và tương tác với người dùng, tiêu biểu gồm có:

- Text, Button, TextField, Image:
 - Các widget cơ bản để hiển thị chữ, nút bấm, ô nhập liệu và hình ảnh. Chúng được thiết kế để hoạt động tốt với hệ thống Material Design mặc định.
- LazyColumn:
 - Đây là thành phần then chốt cho việc hiển thị danh sách lớn (tương tự RecyclerView).
 - Điểm mạnh của LazyColumn là chỉ hiển thị và tái chế các mục (item) đang hiển thị trên màn hình, giúp tối ưu hiệu suất và bộ nhớ một cách tự động, không cần cài đặt adapter phức tạp.
- Material 3 Components:
 - Sử dụng các thành phần cao cấp hơn như Card, TopAppBar, BottomAppBar, FloatingActionButton, v.v., giúp xây dựng các màn hình phức tạp nhanh chóng theo chuẩn thiết kế hiện đại.

4.3 Navigation

Việc điều hướng trong Compose được đơn giản hóa đáng kể so với việc quản lý các Fragment phức tạp.

- Navigation Compose:
 - Đây là thư viện chính thức, tích hợp sâu vào framework Compose, cho phép định nghĩa các "đích đến" (destinations) và xử lý việc chuyển đổi giữa chúng.
- Tư duy "Single Activity → Multi Screen Composables":
 - Đây là sự thay đổi kiến trúc cốt lõi. Trong các ứng dụng Compose hiện đại, toàn bộ ứng dụng thường chỉ sử dụng một Activity duy nhất.
 - Mỗi màn hình được định nghĩa bởi một hàm Composable toàn màn hình. Navigation Component quản lý việc thêm/xóa các Composable này khỏi bộ nhớ (back stack), loại bỏ sự phức tạp của vòng đời (lifecycle) Fragment truyền thống.

4.4 Theming & Material Design

Hệ thống Theming trong Compose giúp tạo giao diện nhất quán và đồng bộ cho toàn ứng dụng.

- **MaterialTheme:**
 - Đây là đối tượng trung tâm quản lý giao diện. Nó chứa đựng 3 thuộc tính chính được sử dụng mặc định bởi tất cả các Material Components khác:
 - **ColorScheme:** Định nghĩa bảng màu chính (Primary, Secondary, Background, Error, v.v.).
 - **Typography:** Định nghĩa hệ thống font chữ (Headlines, Body texts, Captions, v.v.).
 - **Shape:** Định nghĩa hình dạng và bo góc cho các thành phần (nút, card, dialog, v.v.).
- **Dark mode support:**
 - Bằng cách định nghĩa riêng **DarkColorScheme** và sử dụng hàm kiểm tra hệ thống (**isSystemInDarkTheme()**), ứng dụng Compose có thể tự động chuyển đổi giữa chế độ sáng và tối một cách mượt mà chỉ với vài dòng code.

Việc nắm vững các thành phần này là chìa khóa để xây dựng các ứng dụng Android hiệu quả, đẹp mắt và dễ bảo trì bằng Jetpack Compose.

5. Vòng đời Composable và quản lý trạng thái

5.1 Cơ chế vận hành cốt lõi của Jetpack Compose

Cần phân biệt rõ hai loại vòng đời khác nhau mà ứng dụng Android phải xử lý:

- **Activity và Fragment lifecycle:** bao gồm các hàm như **onCreate**, **onStart**, **onResume**,... do hệ thống Android quản lý.
 - **Composition lifecycle:** mô tả quá trình một Composable được đưa vào UI (enter), được cập nhật (recompose) và bị loại bỏ (leave). Compose tự quản lý toàn bộ state nội bộ và các side-effect trong vòng đời này.
- [1]

Với Jetpack Compose, không gọi trực tiếp callbacks của Activity để cập nhật UI. Quá trình recompose thường được kích hoạt khi có thay đổi đối với đối tượng **State<T>**. Compose sẽ theo dõi quá trình thay đổi này, đồng

thời chạy tất cả composable trong Composition có khả năng đọc State<T> và bất kỳ composable nào mà quá trình này gọi.

Side-effects in Compose

Hàm Side-effects thực hiện các hành động làm thay đổi trạng thái của hệ thống bên ngoài hoặc tương tác với môi trường bên ngoài phạm vi của nó.

- *LaunchedEffect* giúp bạn chạy code bất đồng bộ (như gọi API, đếm giờ...) một cách an toàn, gắn liền với vòng đời của giao diện.
- *DisposableEffect*: Thực hiện những xử lý dọn dẹp
- *SideEffect*: Đưa trạng thái của Compose sang các đoạn mã không phải Compose.

Nguyên tắc trong thiết kế compose luôn hủy đăng ký và dọn dẹp tài nguyên ngay khi không còn sử dụng.

Công cụ lưu state:

- *remember*: giữ giá trị state qua các lần recompose nhưng sẽ mất đi khi xoay màn hình.
- *rememberSaveable*: lưu vào SavedInstanceState để tồn tại ngay cả khi recompose và cấu hình thay đổi như xoay (như xoay màn hình, đổi ngôn ngữ). [3]

5.2 Vai trò của ViewModel trong kiến trúc Clean Architecture với Jetpack Compose

Vai trò ViewModel

- Quản lý và cung cấp UI State: Đóng vai trò là nguồn chân lý duy nhất (Single Source of Truth), lưu trữ trạng thái hiển thị và đảm bảo luồng dữ liệu một chiều (Unidirectional Data Flow) để Compose cập nhật giao diện chính xác.
- Tách biệt logic nghiệp vụ: Hoạt động như cầu nối trung gian giữa Presentation Layer và Domain Layer, giúp cô lập mã nguồn giao diện khỏi các logic xử lý nghiệp vụ phức tạp từ Use Cases.
- Bảo toàn dữ liệu theo vòng đời: Đảm bảo tính toàn vẹn của dữ liệu UI trước các thay đổi cấu hình hệ thống (như xoay màn hình) và hỗ trợ quản lý tài nguyên để ngăn chặn rò rỉ bộ nhớ.[1][4]

Cơ chế vận hành

- Luồng dữ liệu: cung cấp UiState (thông qua StateFlow hoặc LiveData) để Composable quan sát và hiển thị.
- Luồng sự kiện: UI không tự xử lý logic, mà gửi sự kiện về ViewModel. ViewModel xử lý nghiệp vụ và cập nhật lại UiState, khép kín vòng lặp.

Lợi ích kỹ thuật:

- Tối ưu kiểm thử: ViewModel độc lập với UI framework, giúp việc Unit Test logic nghiệp vụ dễ dàng và chính xác.
- Kiến trúc được tổ chức tốt: Giảm tải logic cho UI (UI chỉ hiển thị), tách biệt hoàn toàn code giao diện và code xử lý.
- Bảo toàn dữ liệu: Đảm bảo trạng thái UI không bị mất khi thay đổi cấu hình (xoay màn hình), phù hợp tuyệt đối với tư duy lập trình phản ứng (Reactive) của Compose.

5.3 Cơ chế quan sát state và tối ưu hóa tài nguyên

Tiêu chuẩn lựa chọn luồng dữ liệu

- StateFlow: Được khuyến nghị là tiêu chuẩn cho trạng thái UI nhờ đặc tính luôn duy trì giá trị hiện tại.
- LiveData: Vẫn được hỗ trợ trong Compose thông qua adapter chuyển đổi, tuy nhiên StateFlow được ưu tiên hơn trong các dự án Kotlin-first.
- Channel/SharedFlow: Chuyên biệt cho xử lý các sự kiện một lần để tránh việc lưu giữ trạng thái thừa, không phù hợp để làm state chính.

Cơ chế tích hợp và an toàn theo Lifecycle

- Chuyển đổi sang State: Trong Composable, các luồng dữ liệu (Flow/LiveData) phải được chuyển đổi thành State<T> (thông qua collectAsState hoặc observeAsState) để kích hoạt cơ chế Recomposition mỗi khi có dữ liệu mới.
- Tối ưu hóa tài nguyên: Bắt buộc sử dụng các API nhận thức vòng đời như *collectAsStateWithLifecycle()* hoặc pattern *repeatOnLifecycle*. Cơ chế này đảm bảo việc thu thập dữ liệu tự động ngưng khi ứng dụng xuống background, giúp tiết kiệm tài nguyên hệ thống và ngăn chặn các cập nhật UI không an toàn (crash).

III. References

[1] Pathway Jetpack Compose tổng quan các lesson về Compose, lifecycle & architecture):

<https://developer.android.com/courses/pathways/compose?hl=vi>

[2] Codelab: State in Jetpack Compose (hoist state, StateFlow/LiveData, produceState):

<https://developer.android.com/codelabs/jetpack-compose-state?hl=vi>

[3] Compose tutorial & effects (API: remember, rememberSaveable, LaunchedEffect, DisposableEffect, produceState, snapshotFlow):

<https://developer.android.com/develop/ui/compose/tutorial?hl=vi>

[4] Lifecycle & ViewModel docs (repeatOnLifecycle, SavedStateHandle, lifecycle-aware patterns, collectAsStateWithLifecycle helper):

<https://developer.android.com/topic/libraries/architecture/lifecycle>

[5] Google Developers. *Build adaptive apps with Jetpack Compose*. URL: <https://developer.android.com/develop/ui/compose/build-adaptive-apps>